Universidad Tecnológica del Perú

DISEÑO DE PATRONES

DOCENTE:
Mg. Iván Robles Fernández

MOTIVACIÓN

Universidad Tecnológica del Perú

CONOCIMIENTOS PREVIOS

¿Qué entiendes por principios de diseño?

LOGRO DE LA SESIÓN

Al finalizar la unidad el participante identifica los principios de diseño de los patrones de software para la solución de problemas

IMPORTANCIA DE LA SESIÓN

La aplicación de los patrones de software a permitirá solucionar los problemas habituales de software planteados.

TEMA :

- Introducción a principios de diseño.
- Introducción a principios SOLID.
- Principio de Responsabilidad Única

Introducción a principios de diseño

Son fundamentos clave que guían la creación de software robusto, flexible y mantenible. Estos principios ayudan a los desarrolladores a evitar problemas comunes y a mejorar la calidad del código. A continuación, vemos los principios más importantes:

1. Responsabilidad Única (Single Responsibility Principle - SRP)
2. Abierto/Cerrado (Open/Closed Principle - OCP)
3. Sustitución de Liskov (Liskov Substitution Principle - LSP)
4. Segregación de Interfaces (Interface Segregation Principle - ISP)
5. Inversión de Dependencia (Dependency Inversion Principle - DIP)

Introducción a principios de diseño

Los principios de diseño en programación orientada a objetos se enfocan en crear sistemas que sean fáciles de mantener y expandir. Algunos de los principios de diseño más importantes incluyen:

- **Encapsulamiento**: Ocultar los detalles internos de los objetos y exponer solo lo necesario.
- **Modularidad**: Dividir un sistema en módulos independientes que puedan desarrollarse y probarse de manera aislada.

Introducción a principios de diseño

- **Abstracción**: Simplificar la complejidad ocultando detalles innecesarios y exponiendo solo las funcionalidades esenciales.

- **Herencia**: Permitir que nuevas clases hereden propiedades y métodos de clases existentes, promoviendo la reutilización de código.

- **Polimorfismo**: Permitir que una interfaz se comporte de diferentes maneras dependiendo del contexto.

Introducción a principios de diseño

1. **Principio de Responsabilidad Única (Single Responsibility Principle - SRP)**

   Una clase debe tener una única responsabilidad, es decir, una única razón para cambiar. Esto ayuda a mantener el código más modular y fácil de mantener.

Introducción a principios de diseño: Ejemplo

1. **Principio de Responsabilidad Única (Single Responsibility Principle - SRP)**

Supongamos que estamos desarrollando una aplicación para gestionar una biblioteca. Inicialmente, podríamos tener una clase Libro que almacena los detalles del libro y también maneja el préstamo. Sin embargo, esto viola el SRP

Introducción a principios de diseño: Ejemplo

```
class Libro {
            private String titulo;
            private String autor;
 public Libro(String titulo, String autor) {
         this.titulo = titulo;
         this.autor = autor;
 }

 public void prestar() {
        // Lógica para prestar el libro
 }

 public void devolver() {
       // Lógica para devolver el libro
 }
}
```

```
// Corrección:
class Libro {
      private String titulo;
      private String autor;

      public Libro(String titulo, String autor) {
              this.titulo = titulo;
              this.autor = autor;
   }
 }
```

Introducción a principios de diseño: Ejemplo

```
// Ejercicio:
class Usuario {
      private String nombre;
      public Usuario(String nombre) {
            this.nombre = nombre;
  }

    public void registrarUsuario() {
     // Lógica para registrar usuario
  }

 public void prestarLibro(Libro libro) {
     // Lógica para prestar libro
  }
}
```

// ¿Qué problemas encuentras en la clase Usuario respecto al SRP?

// Respuesta:

La clase Usuario está manejando tanto el registro de usuarios como la lógica de préstamo de libros, lo que viola el SRP.

Es recomendable separar estas responsabilidades en clases diferentes.

Introducción a principios de diseño:

2. Principio de Abierto/Cerrado (Open/Closed Principle - OCP)

Las entidades de software deben estar abiertas para su extensión, pero cerradas para modificación. Esto significa que el comportamiento de una entidad puede ser extendido sin modificar su código fuente.

Introducción a principios de diseño: Ejemplo

2. Principio de Abierto/Cerrado (Open/Closed Principle - OCP)

Un sistema de pago en línea que inicialmente implementa el pago con tarjeta de crédito. Más tarde, queremos añadir la opción de pago con PayPal sin modificar el código existente..

Introducción a principios de diseño: Ejemplo

```
abstract class Pago {
        public abstract void realizarPago();
}

class PagoConTarjeta extends Pago {
     @Override
      public void realizarPago() {
             // Lógica de pago con tarjeta
   }
}

class PagoConPayPal extends Pago {
     @Override
public void realizarPago() {
        // Lógica de pago con PayPal
   }
}
```

```
// Ejercicio:
class PagoConCriptomoneda extends Pago {
     @Override
public void realizarPago() {
      // Lógica de pago con criptomonedas
     }
}
```

// ¿Cómo se aplica el OCP en esta situación?

// Respuesta:

El OCP se aplica al permitir la extensión del comportamiento de la clase Pago sin modificar las clases existentes. Solo se añade una nueva clase PagoConCriptomoneda que implementa la lógica de pago con criptomonedas.

Introducción a principios de diseño:

3. Principio de Sustitución de Liskov (Liskov Substitution Principle - LSP)

Los objetos de una clase derivada deben ser reemplazables por objetos de la clase base sin alterar el funcionamiento correcto del programa. Esto garantiza que las clases derivadas extienden la funcionalidad de la clase base sin cambiar su comportamiento esencial.

Introducción a principios de diseño: Ejemplo

3. Principio de Sustitución de Liskov (Liskov Substitution Principle - LSP)

Si tenemos una clase base Animal y una clase derivada Perro, cualquier instancia de Perro debería poder reemplazar a una instancia de Animal sin problemas.

Introducción a principios de diseño: Ejemplo

```
abstract class Animal {
      public abstract void hacerSonido();
}
class Perro extends Animal {
      @Override
      public void hacerSonido() {
            System.out.println("Ladrido");
      }
}
class Gato extends Animal {
     @Override
     public void hacerSonido() {
            System.out.println("Maullido");
   }
}
```

```
// Ejercicio:
class Pez extends Animal {
     @Override
     public void hacerSonido() {
     throw new UnsupportedOperationException("Los peces no hacen sonido");
   }
}

// ¿Viola Pez el LSP?
```

// Respuesta:

Sí, la clase Pez viola el LSP porque no puede sustituir a la clase base Animal sin alterar el comportamiento esperado, ya que los peces no pueden hacer sonido.

Introducción a principios de diseño: Ejemplo

4. Principio de Segregación de Interfaces (Interface Segregation Principle - ISP)

Los clientes no deben estar obligados a depender de interfaces que no utilizan. Es preferible tener varias interfaces específicas en lugar de una única interfaz general.

Introducción a principios de diseño: Ejemplo

4. Principio de Segregación de Interfaces (Interface Segregation Principle - ISP)

Supongamos un sistema de impresión donde tenemos una interfaz Impresora con métodos imprimir, escanear, y faxear. No todos los dispositivos necesitan implementar todos estos métodos.

Introducción a principios de diseño: Ejemplo

```
interface Impresora {
    void imprimir();
    void escanear();
    void faxear();
}

class ImpresoraSimple implements Impresora
{
    @Override
    public void imprimir() {
        // Lógica para imprimir
    }

    @Override
    public void escanear() {
        // No implementado
    }

    @Override
    public void faxear() {
        // No implementado
    }
}
```

```
// Ejercicio:

interface Impresora {
    void imprimir();
}

interface Escaner {
    void escanear();
}
```

Introducción a principios de diseño: Ejemplo

```
interface Impresora {
    void imprimir();
    void escanear();
    void faxear();
}

class ImpresoraSimple implements Impresora
{
    @Override
    public void imprimir() {
        // Lógica para imprimir
    }

    @Override
    public void escanear() {
        // No implementado
    }
```

```
interface Fax {
    void faxear();
}

class ImpresoraMultifuncional implements Impresora,
Escaner, Fax {
    @Override
    public void imprimir() {
        // Lógica para imprimir
    }
}
```

Introducción a principios de diseño: Ejemplo

```
    @Override
    public void faxear() {
        // Lógica para faxear
    }
}

// ¿Cómo se aplica el ISP en este ejemplo?
```

// Respuesta:

El ISP se aplica dividiendo una interfaz general en interfaces específicas. De esta manera, las clases implementan solo las interfaces que necesitan, evitando depender de métodos que no usan.

Introducción a principios de diseño:

5. **Principio de Inversión de Dependencia (Dependency Inversion Principle - DIP)**

Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones. Las abstracciones no deben depender de los detalles, los detalles deben depender de las abstracciones.

Introducción a principios de diseño: Ejemplo

5. **Principio de Inversión de Dependencia (Dependency Inversion Principle - DIP)**

En una aplicación de gestión de pedidos, en lugar de que la clase Pedido dependa directamente de una clase concreta BaseDatos, puede depender de una interfaz IBaseDatos.

Introducción a principios de diseño: Ejemplo

```
interface IBaseDatos {
    void guardarPedido(Pedido pedido);
}

class BaseDatosSQL implements IBaseDatos
{
    @Override
    public void guardarPedido(Pedido pedido) {
        // Lógica para guardar en SQL
    }
}

class Pedido {
    private IBaseDatos baseDatos;

    public Pedido(IBaseDatos baseDatos) {
        this.baseDatos = baseDatos;
    }
    public void procesarPedido() {
        baseDatos.guardarPedido(this);
    }
}
```

// Ejercicio:

```
class BaseDatosNoSQL implements IBaseDatos {
    @Override
    public void guardarPedido(Pedido pedido) {
        // Lógica para guardar en NoSQL
    }
}

// ¿Qué ventaja ofrece el DIP en esta situación?
```

Introducción a principios de diseño: Ejemplo

// Respuesta:

El DIP permite cambiar la implementación de la base de datos (por ejemplo, de SQL a NoSQL) sin modificar la clase Pedido, ya que esta depende de la abstracción IBaseDatos y no de una implementación concreta.

DISEÑO DE PATRONES

Introducción a principios de diseño: Casos Practicos

Caso Práctico 1: Principio de Responsabilidad Única - SRP

Contexto: Un sistema de gestión de una biblioteca donde es necesario manejar la información de los libros y gestionar su impresión y almacenamiento en la base de datos.

Problema: La clase Libro tiene múltiples responsabilidades: almacenar información y gestionar la impresión.

DISEÑO DE PATRONES

Introducción a principios de diseño: Casos Practicos

Caso Práctico 2: Principio de Responsabilidad Única - SRP

Contexto: Un sistema de gestión de empleados donde cada empleado tiene que calcular su salario y generar su recibo de pago.

Problema: La clase Empleado realiza tanto el cálculo del salario como la impresión del recibo.

Universidad Tecnológica del Perú

Introducción a principios SOLID

Los principios SOLID son un subconjunto de principios de diseño orientados a mejorar la robustez y la flexibilidad del software. Son cinco principios específicos:

- **Single Responsibility Principle (SRP)**: Una clase debe tener una sola responsabilidad o motivo para cambiar.

- **Open/Closed Principle (OCP)**: Las entidades de software deben estar abiertas para extensión, pero cerradas para modificación.

Universidad Tecnológica del Perú

Introducción a principios SOLID

- **Liskov Substitution Principle (LSP)**: Las clases derivadas deben poder sustituir a sus clases base sin alterar el comportamiento del programa.

- **Interface Segregation Principle (ISP)**: Los clientes no deben estar obligados a depender de interfaces que no usan.

- **Dependency Inversion Principle (DIP)**: Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

Deferencias entre Introducción a principios diseño y Principios SOLID

Alcance y Especificidad:

- **Principios de Diseño**: Son más generales y abarcan una amplia gama de prácticas de diseño orientadas a objetos.

- **Principios SOLID**: Son más específicos y se centran en la creación de código más robusto y fácil de mantener.

Deferencias entre Introducción a principios diseño y Principios SOLID

Objetivos:

- **Principios de Diseño**: Promueven buenas prácticas generales de diseño para mejorar la calidad del software.

- **Principios SOLID**: Abordan problemas específicos en el diseño de software para mejorar la escalabilidad y la flexibilidad del código.

Deferencias entre Introducción a principios diseño y Principios SOLID

Aplicación:

- **Principios de Diseño**: Pueden aplicarse a lo largo de todo el ciclo de desarrollo del software.

- **Principios SOLID**: A menudo se utilizan en etapas específicas del desarrollo para abordar problemas particulares.

Principio de Responsabilidad Única (SRP)

Establece que una clase debería tener una única responsabilidad o razón para cambiar. En otras palabras, cada clase debe tener una sola tarea o función dentro del software, lo que facilita el mantenimiento y la comprensión del código

Principio de Responsabilidad Única (SRP)

Importancia:

- **Facilita el Mantenimiento:** Las clases con una única responsabilidad son más fáciles de entender, mantener y modificar.

- **Mejora la Reutilización:** Las clases con una sola responsabilidad son más fáciles de reutilizar en diferentes contextos porque no están acopladas a otras funcionalidades.

Principio de Responsabilidad Única (SRP)

Importancia:

- **Reduce el Acoplamiento:** Al tener clases con responsabilidades bien definidas, se reduce la dependencia entre ellas, lo que facilita el aislamiento y la prueba de componentes individuales.

- **Aumenta la Cohesión:** Las clases que hacen una sola cosa tienden a tener una alta cohesión, lo que significa que los métodos y atributos de la clase están estrechamente relacionados entre sí.

Principio de Responsabilidad Única (SRP)

Este principio nos viene a decir que una clase sólo debería tener una única razón para cambiar.

"Una clase debe tener una única razón para cambiar."

En términos prácticos, este principio establece que:

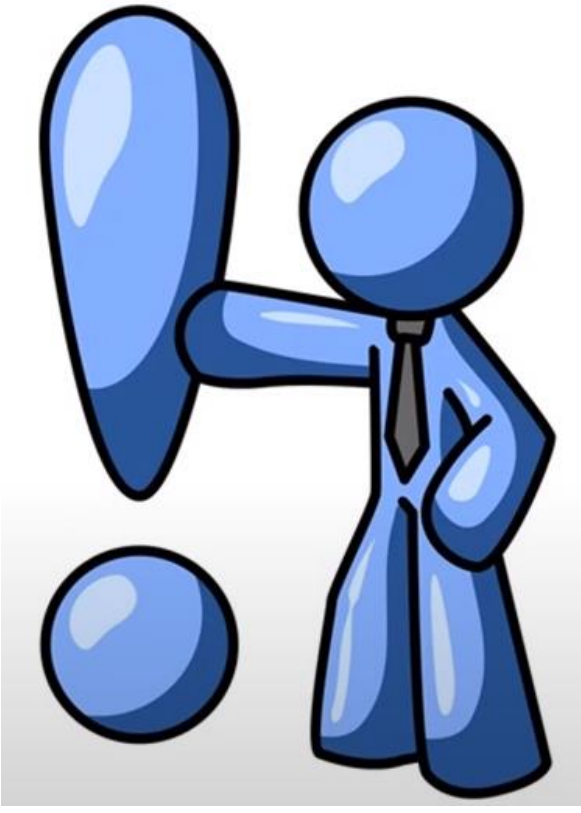
- **"Una clase debe tener una y solo una única causa por la cual puede ser modificada."**
- **"Cada clase debe ser responsable de realizar una actividad del sistema**

Principio de Responsabilidad Única (SRP) ¿Porque usar ?

- Es más fácil re-utilizar partes del código
- Las clases grandes son mas difíciles de leer y cambiar.
- Solucionamos el dilema del nombre de la clase

CONCLUSIONES

- Los principios de diseño proporcionan guías para crear software de alta calidad, mantenible y escalable.
- Los principios SOLID, son cinco reglas fundamentales para el diseño orientado a objetos: **S**ingle Responsibility, **O**pen/Closed, **L**iskov Substitution, **I**nterface Segregation, y **D**ependency Inversion.
- El Principio de Responsabilidad Única (SRP) establece que una clase debe tener una sola responsabilidad o motivo de cambio, facilitando el mantenimiento y la evolución del software.
- Implementar estos principios mejora la cohesión, reduce el acoplamiento y aumenta la flexibilidad del código.



Universidad Tecnológica del Perú

FACULTADAD DE INGENIERÍA

METACOGNICIÓN

1 ¿ QUÉ HE APRENDIDO ?

SER CONSCIENTE

2 ¿ CÓMO LO HE APRENDIDO ?

DESCRIBIR

3 ¿ QUÉ HA RESULTADO MÁS FÁCIL, MÁS DIFICIL O MÁS NOVEDOSO ?

EVALUAR

4 ¿ CÓMO LO PUEDO MEJORAR? ¿PARA QUÉ ME HA SERVIDO?

PLANIFICAR

Universidad Tecnológica del Perú

FACULTADAD DE INGENIERÍA

¡GRACIAS!

EN LA UTP SABEMOS QUE TODO ES POSIBLE